학생 배포용

ERICA 캠퍼스 자율주행 과제

실행 방법 · 주행 경로 · 평가 방법

카메라 영상을 OpenCV(필요하면 CNN)로 처리하고 ROS2로 차량을 움직여, 시뮬레이터 속 한양대 ERICA 캠퍼스를 한 바퀴(1962 m) 자율주행합니다. 판마다 신호등, 무단횡단자, 끼어드는 차, 공사 콘, 어린이보호구역 같은 **미션 8개가 무작위로** 놓입니다. 위치를 외우거나 하드코딩해서는 통과할 수 없고, 카메라로 보고 판단해야 합니다.

팀	기간	코스	미션
2~3명	**4주**	**1962 m**	**판마다 8개**
다른 팀과 코드 공유 금지	배포일 기준, 5주차 발표	신호 정지선 7곳, 제한 1200 s	종류·순서·위치 무작위

시점	할 일	성적
0주차 (배포)	환경 설치, 뼈대 코드로 한 바퀴 도는지 확인	없음
2주차 말	**중간 점검:** 이벤트 없는 판에서 신호를 지키며 완주	10%
4주차 말	**최종 제출:** 코드(`team_XX.zip`), 보고서 PDF, 주행 영상	주행 60% + 보고서 20%
5주차	발표	발표·영상 10%

1 실행 방법 | 2 주행 경로와 미션 | 3 평가 방법 | 4 제출과 규칙 | 부록 파일 위치

토픽 필드처럼 더 자세한 내용은 과제 저장소의 `docs/assignment/student-guide.md` 에 있습니다. 이 문서와 안내서가 다르면 안내서를 따릅니다.

1 실행 방법

1.1 필요한 환경

항목	요구 사항
OS	Ubuntu 22.04 (WSL, macOS, Windows는 지원하지 않습니다)
ROS	ROS2 Humble (`/opt/ros/humble`)
GPU	NVIDIA GPU와 드라이버. 카메라 영상을 GPU로 렌더링합니다
Python	시스템 Python 3.10 (`/usr/bin/python3`)

> **conda를 끄고 쓰세요**
> conda가 켜진 터미널에서는 ROS 명령이 깨집니다. 터미널을 열 때마다 `conda deactivate` 를 먼저 하세요.

실습실 PC와 개인 PC 모두 쓸 수 있습니다. 채점 PC는 Ubuntu 22.04, NVIDIA RTX 6000 Ada(48 GB)입니다.

1.2 설치 (처음 한 번)

```
git clone https://github.com/ingyu0808/avsim-assignment.git avsim-assignment
cd avsim-assignment
./scripts/get-sim.sh                    # 시뮬레이터를 받아 UnitySim/Build/Linux에 풂
./ros2_ws/setup_deps.sh                 # apt 의존성 설치 (sudo 필요)
source /opt/ros/humble/setup.bash
cd ros2_ws && colcon build --symlink-install && cd ..
./scripts/new-team-package.sh 07        # 팀 번호 두 자리 → ros2_ws/src/team_07
cd ros2_ws && colcon build --symlink-install --packages-select team_07 && cd ..
```

`team_07`은 뼈대 패키지 `student_driver`를 팀 이름으로 복사한 것입니다. 앞으로 이 패키지만 고치고, 이 패키지를 제출합니다. 시뮬레이터는 크기 때문에 저장소에 없고, `get-sim.sh`가 저장소의 GitHub Release에서 받습니다. 받지 못하면 Releases 페이지에서 `avsim-sim-<버전>.tar.gz`를 브라우저로 받아 `./scripts/get-sim.sh <받은 파일>`로 푸세요. 수정본 공지가 오면 `git pull` 뒤 `get-sim.sh`와 `colcon build`를 다시 실행합니다. `ros2_ws/src/team_XX`는 저장소에 없는 폴더라 pull과 겹치지 않습니다.

1.3 실행 (터미널 2개)

두 터미널 모두 저장소 폴더에서 먼저 `source /opt/ros/humble/setup.bash && source ros2_ws/install/setup.bash`를 실행합니다.

```
# 터미널 1: 시뮬레이터 (ROS 브릿지는 자동으로 뜹니다)
./scripts/run-sim.sh

# 터미널 2: 팀 주행 스택
ros2 launch team_07 drive.launch.py
```

뼈대를 그대로 실행하면 차가 신호와 사람을 무시하고 5 m/s로 한 바퀴를 돈 뒤 정문에 멈춥니다. 설치 직후 이것부터 확인하세요.

1.4 시뮬레이터 옵션

명령	뜻
`./scripts/run-sim.sh`	매번 새 무작위 판
`./scripts/run-sim.sh --seed 48213`	같은 판을 다시 (실패한 판 디버깅용)
`./scripts/run-sim.sh --no-scenario`	이벤트 없는 판 (중간 점검과 같은 조건)
`./scripts/run-sim.sh --result /tmp/run.json`	판이 끝나면 결과 파일을 이 경로에 씀
`./scripts/run-sim.sh --headless`	창 없이 실행 (모니터가 없는 PC, GPU는 필요)

- **지금 판의 시드:** 판이 만들어질 때마다 플레이어 로그에 시드가 찍힙니다. 뼈대는 출발할 때 한 번 리셋하므로 **마지막 줄**이 지금 판입니다. 결과 파일에도 시드가 들어 있습니다.

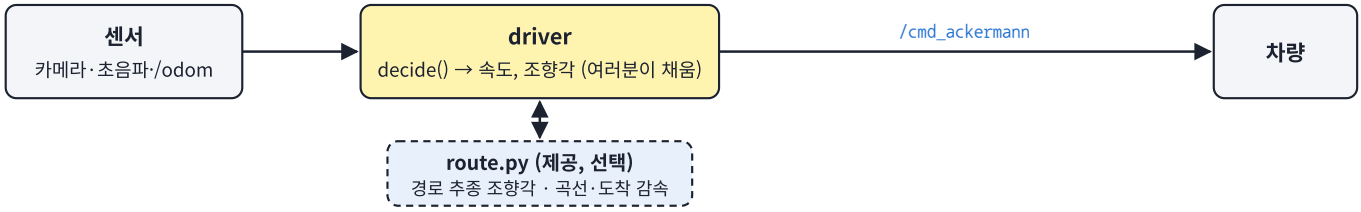
  ```
  grep "\[Scenario\] seed" ~/.config/unity3d/DefaultCompany/UnitySim/Player.log | tail -1
  ```

- **리셋:** `ros2 service call /reset_simulation std_srvs/srv/Trigger`를 부르면 차량이 정문으로 돌아가고 새 시드로 판이 다시 만들어집니다. `--seed`로 실행했다면 같은 판이 다시 만들어집니다.

1.5 뼈대 코드 구조

파일 (ros2_ws/src/team_07/ 안)	역할
team_07/driver.py	**여러분이 채울 곳.** /cmd_ackermann 을 직접 보냄
team_07/route.py	제공 도우미 (선택). 경로 추종 조향각, 곡선·도착 감속
team_07/control.py	route.py의 속도 규칙 (테스트 포함)
launch/drive.launch.py	제출할 실행 파일
setup.py , package.xml , test/	패키지 설정과 테스트

- **driver (채울 곳):** 센서 구독, 영상 변환(to_bgr), 20 Hz 명령 발행까지 되어 있습니다. decide() 가 (속도 m/s, 조향각 rad) 를 돌려주면 그대로 /cmd_ackermann 으로 나갑니다. 지금은 route.py 값을 그대로 돌려줘 신호와 사람을 무시합니다.
- **route.py (제공, 선택):** /odom 으로 고정 경로를 따라가는 조향각과 곡선·도착 감속 속도를 계산하고, 출발할 때 차를 정문으로 리셋합니다. 쓰지 않고 차선을 직접 보고 조향해도 됩니다.
- 노드를 더 만들어도 됩니다. 단, 실행은 drive.launch.py 하나로 되어야 하고 /cmd_ackermann 은 한 노드만 보냅니다. speed:=4.0 처럼 순항 속도를 바꿀 수 있습니다(최대 6 m/s).



여러분의 노드가 속도와 조향각을 정해 /cmd_ackermann 으로 직접 보냅니다. 0.5 s 넘게 명령이 없으면 차가 섭니다.

1.6 ROS 토픽 요약

입력 토픽	타입	주기	설명
/camera/image_raw	sensor_msgs/Image	20 Hz	전방 카메라, rgb8 640×480
/camera/camera_info	sensor_msgs/CameraInfo	20 Hz	fx = fy = 320, 수평 화각 90°
/odom	nav_msgs/Odometry	30 Hz	시뮬레이터 정답 위치 (x = 북, y = 서)
/imu/data	sensor_msgs/Imu	100 Hz	
/ultrasonic/front 외 3개	sensor_msgs/Range	20 Hz	front, rear, left, right. 최대 4 m

출력 토픽	타입	설명
/cmd_ackermann	ackermann_msgs/AckermannDrive	차량 제어. speed (m/s), steering_angle (rad, 양수 = 왼쪽, 최대 ±0.6). 한 노드만 발행. 0.5 s 안에 새 명령이 없으면 차가 멈춤
/team/route_state	std_msgs/String (JSON)	뼈대가 내는 디버깅용 상태: 진행 거리, 경로 이탈 거리, 상태. 채점에는 안 씀

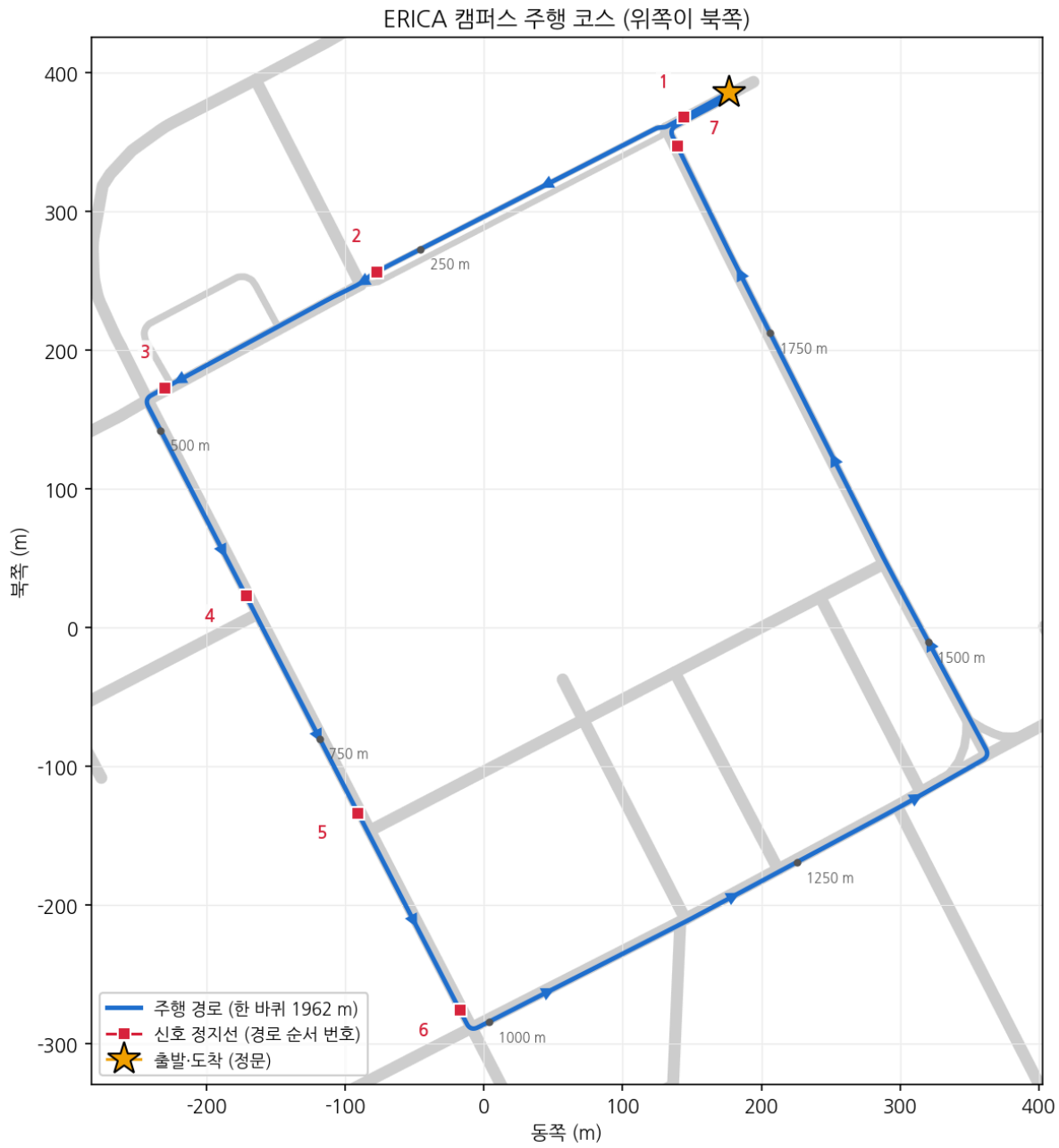
신호등은 카메라로만 찾습니다

신호등의 **위치와 색**은 어떤 토픽으로도 주지 않습니다. 카메라 영상에서 신호등을 찾고 색을 읽어야 합니다.

1.7 문제가 생기면

증상	해결
창은 떴는데 `ros2 topic list` 에 `/camera/image_raw` 가 없다	브릿지가 굳은 상태입니다. `systemctl --user restart avsim-ros-bridge`
`ros2 topic echo` 가 `!rclpy.ok()` 오류를 낸다	ROS 데몬 문제입니다. `ros2 daemon stop && ros2 daemon start`
차가 서 있고 `/cmd_ackermann` 이 안 나온다	주행 노드가 죽었습니다. 터미널 2의 오류를 확인하세요
`RESET_POSE_MISMATCH` 에서 멈춘다	다른 프로그램이 같은 ROS 도메인에 `/odom` 을 내고 있습니다. `ROS_DOMAIN_ID` 를 확인하세요
창이 검게 멈춘다 (모니터 없는 PC)	`--headless` 로 실행하세요

2 주행 경로와 미션



파란 선이 주행 경로, 빨간 네모가 신호 정지선(숫자 = 경로 순서), 별이 출발·도착 지점(정문)입니다. 위쪽이 북쪽입니다.

- **경로:** 정문에서 출발해 반시계 방향으로 캠퍼스를 한 바퀴 돌아 정문으로 돌아옵니다(1962 m). 경로는 매번 같고, 뼈대의 `route.py`가 따라가는 조향각을 계산해 줍니다.
- **완주:** 경로 끝(출발 지점 5 m 안)에 **멈추면** 완주입니다. 뼈대가 알아서 멈춥니다.
- **시간 제한:** 1200 s. 넘기면 그 판은 0점입니다. 신호와 미션 대기 시간이 있으므로, 달리는 동안은 적어도 2.5 m/s를 내는 것이 안전합니다.

2.1 구간별 안내

진행 거리는 `/team/route_state`의 `progress_m`과 같은 값입니다. 신호 정지선은 모두 교차로에 들어가기 전에 있고, 교차로를 건넌 뒤에는 정지선이 없습니다.

구간	진행 거리	지도에서	신호 정지선 (번호: 진행 거리)
출발	0–60 m	정문(별)에서 출발	1: 38 m
직선 ①	60–470 m	위쪽 변을 따라 왼쪽으로	2: 286 m, 3: 460 m
좌회전	470–480 m	왼쪽 위 모서리	없음
직선 ②	480–981 m	왼쪽 변을 따라 아래로	4: 634 m, 5: 810 m, 6: 970 m
좌회전	981–990 m	아래쪽 모서리	없음
직선 ③	990–1402 m	아래쪽 변을 따라 오른쪽으로	없음
좌회전	1402–1412 m	오른쪽 아래 모서리	없음
직선 ④	1412–1908 m	오른쪽 변을 따라 위로	7: 1900 m
우회전·도착	1908–1962 m	정문으로 돌아옴	없음

2.2 미션 8종

판마다 아래 8종 가운데 **8개**가 경로 위에 무작위로 놓입니다. 종류, 순서, 위치, 보행자 외형, 타이밍이 모두 달라집니다.

계열	미션	상황	해야 할 일
양보	늦게 건너는 사람	신호 횡단보도에서 초록불이 된 뒤 0.5~3 s 늦게 건너기 시작	초록불이어도 사람이 빠질 때까지 기다린다
양보	무단횡단	횡단보도가 아닌 곳에서, 차가 25~40 m 앞에 왔을 때 건넘	멈춘다
양보	신호 없는 횡단보도	신호 없는 횡단보도를 1~3명이 건넘	멈춘다
양보	가려진 아이	갓길 주차 차량 뒤에서, 차가 15~25 m 앞에 왔을 때 아이가 뛰어나옴	멈춘다. 주차 차량이 보이면 미리 감속
주의 분산	서 있기만 하는 사람	신호 횡단보도 앞 연석에 서 있고 건너지 않음	그대로 지나간다 (괜히 서면 감점)
장애물	끼어드는 차	앞에 차가 나타나 4 m/s로 가다 3~8 s 멈췄다 다시 감	따라가다 멈추고, 출발하면 따라간다
장애물	정차 차량·공사 콘	차로 일부를 막은 정차 차량, 또는 콘 5개. 5~15 s 뒤 치워짐	20 m 안쪽에서 멈춘다
표지판	어린이보호구역	시작 표지판부터 해제 표지판까지 150~300 m, 노면이 빨갛다	구간 안에서 10 km/h(2.78 m/s) 이하

배치 규칙

- 계열마다 적어도 하나씩 나오고, 같은 종류는 한 판에 최대 2개입니다.
- 미션끼리는 80 m 이상 떨어져 있습니다.
- 출발 50 m와 마지막 30 m에는 미션이 없습니다.

판 전체에 붙는 조건 (각각 50% 확률)

- **인도 보행자 증가:** 인도에 사람이 24명 더 걷습니다. 인도의 사람 때문에 서면 안 됩니다.
- **낮은 해:** 역광과 긴 그림자가 생깁니다.

2.3 차량 카메라로 본 미션

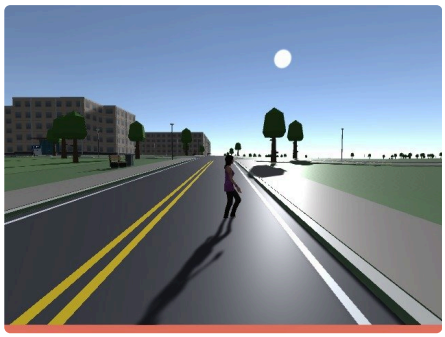
뼈대가 미션을 무시하고 지나가며 찍은 `/camera/image_raw` 화면(640×480)입니다.



늦게 건너는 사람
신호 횡단보도에서 기다리다 초록불 뒤에 건넌다



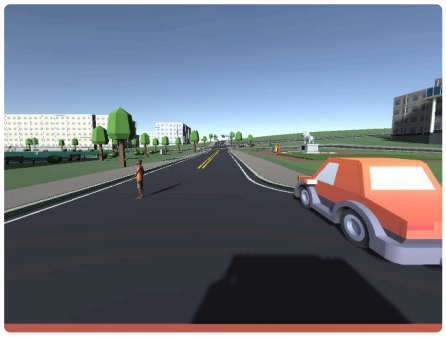
서 있기만 하는 사람
초록불이면 그대로 지나간다



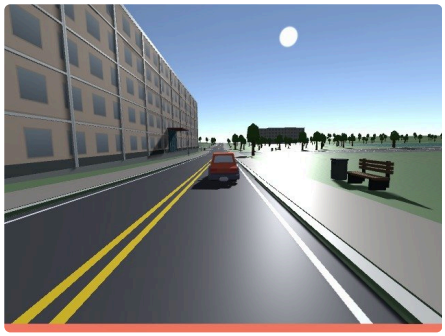
무단횡단
낮은 해 조건



신호 없는 횡단보도
건너는 두 사람



가려진 아이
주차 차량 옆에서 아이가 나온다



끼어드는 차
낮은 해 조건



정차 차량
차로를 막고 서 있다



공사 콘
차로를 비스듬히 막은 콘 5개



어린이보호구역
시작 표지판과 빨간 노면

알아 두면 좋은 것

- **보행자:** 사실적인 사람 모델 12종(어른 9, 아이 3)이고 인도를 걷는 사람도 같은 모델입니다. COCO로 학습한 검출기가 "person"으로 잡습니다.
- **모델 크기:** 작은 모델(예: YOLOv8n)은 15 m 밖의 보행자를 자주 놓칩니다. 5 m/s에서 멈추려면 그보다 먼저 봐야 합니다. 거리별 검출률을 직접 재 보세요.
- **차량과 콘:** 차량(주차, 끼어들기, 정차)은 모두 같은 주황빛 빨간색 로우폴리 세단이고, 콘은 흰 띠가 있는 주황색 원뿔입니다. CNN이 "car"로 잘 못 잡을 수 있으니 색, 형태, 초음파를 함께 쓰세요.
- **초음파는 4 m까지만** 봅니다. 양보 위반은 차 앞 8 m 안에 사람이 있는데 움직이면 걸리므로, 천천히 달린다고 안전하지 않습니다. 카메라로 먼저 봐야 합니다.

3 평가 방법

3.1 성적 구성

항목	비중	계산
중간 점검 (2주차 말)	10%	이벤트 없는 판(`--no-scenario`) 1회의 점수(100점 만점) × 0.1
최종 주행 (4주차 말)	60%	교수자만 아는 **시드 3개** 점수의 평균 × 0.6. 모든 팀이 같은 시드 3개로 채점받습니다
보고서	20%	3.5의 채점표
발표와 주행 영상 (5주차)	10%	3.5 참고

3.2 채점 진행 방식

1. 교수자가 채점 PC에서 `team_XX.zip` 을 깨끗한 워크스페이스에 풀고 `colcon build` (`--symlink-install` 없이)로 빌드합니다. `requirements.txt` 가 있으면 팀별 가상환경에 설치합니다.
2. 시드마다 시뮬레이터를 새로 띄우고 `ros2 launch team_XX drive.launch.py` 를 실행합니다. 사람이 손대지 않습니다.
3. 시뮬레이터는 **채점 모드**로 뜹니다. 창 없이 GPU로 렌더링하고, 신호 정답 토픽을 발행하지 않습니다.
4. 판이 끝나면(도착 정지 또는 1200 s 초과) 시뮬레이터가 결과 파일을 쓰고, 그 점수를 그대로 씁니다.

이 과정은 저장소의 `scripts/grade.sh` 와 같습니다(3.4). 채점 전에 같은 조건으로 직접 돌려 보세요. 중간 점검은 미션이 없으므로 신호 위반, 불필요 정지, 완주 여부가 점수를 가릅니다.

3.3 한 판의 점수

한 판은 100점에서 시작해 위반마다 깎입니다. 0점 아래로는 내려가지 않습니다. 판정 기준은 모두 공개합니다.

항목	기준	점수
충돌	미션 보행자, 차량, 콘과 접촉	**그 판 0점**
미완주	시간 초과(1200 s) 또는 도착 지점에 서지 않음	**그 판 0점**
양보 위반	사람이 차량 앞 0~8 m, 차폭 ±1 m(중심에서 1.875 m) 안에 있는데 0.5 m/s 넘게 움직임. 사람마다 한 번	**−20**
신호 위반	빨강이나 노랑일 때 정지선 통과. 정지선마다	**−20**
보호구역 과속	구간 안에서 3.0 m/s 초과가 1 s 이상. 구간에 들어갈 때마다 한 번	**−10**
불필요 정지	정당한 이유 없이 2 s 넘게 정지(0.15 m/s 미만). 정지할 때마다 한 번	**−5**
근접	걷는 중이거나 도로 위에 있는 보행자와 1.5 m 안으로 접근. 사람마다 한 번	**−5**

정당한 정지 (감점하지 않음)

- 12 m 안에 빨강·노랑 신호 정지선이 있음
- 15 m 안의 도로 위에, 또는 건너는 중인 사람이 있음
- 15 m 안의 차로에 차량·콘이 있음
- 20 m 안에서 장애물 미션이 길을 막고 있음
- 마지막 30 m (도착)

시간 점수는 없습니다. 1200 s 안에만 들어오면 빨리 달려도 더 받지 않습니다.

3.4 스스로 채점하기

결과 파일에는 시드, 이번 판의 미션 목록, 위반 내역(시각과 위치), 보행자 최소 거리, 점수가 들어 있습니다. 판이 끝날 때(도착 정지, 리셋, 시뮬레이터 종료, 시간 초과) 파일이 생깁니다.

```
# 한 판 돌리고 결과 보기
./scripts/run-sim.sh --result /tmp/run.json
/usr/bin/python3 scripts/scenario_average.py /tmp/run.json

# 채점과 같은 조건으로 여러 시드를 돌려 평균 내기 (창 없음, 신호 정답 토픽 꺼짐)
./scripts/grade.sh --launch "ros2 launch team_07 drive.launch.py" 11 22 33
./scripts/grade.sh --launch "ros2 launch team_07 drive.launch.py" --no-scenario   # 중간 점검 조건
```

`grade.sh` 는 결과를 `/tmp/avsim-grade-<날짜-시각>/` 에 남깁니다(`--out DIR` 로 바꿀 수 있습니다). 시드별 결과 `<시드>.json` , 여러분 노드의 출력 `<시드>-student.log` , 점수 요약 `summary.txt` 가 생깁니다.

학습 데이터를 모으려면 주행 중에 `/usr/bin/python3 scripts/record_frames.py ~/frames` 를 실행하세요. 카메라 영상이 0.5 s마다 JPEG로 저장되고, 파일 이름에 진행 거리가 붙습니다.

3.5 보고서와 발표

보고서 (PDF)	점수	내용
인식 방법과 선택 근거	8	신호, 보행자, 차량·콘, 표지판을 각각 어떻게 인식하고 판단했는지. 왜 그 방법을 골랐는지
직접 측정한 성능	6	예: 거리별 보행자 검출률, 신호 색 인식 정확도, 처리 시간. 측정 방법과 표·그래프
실패 분석과 개선	6	실패한 시드와 결과 파일의 위반 내역, 원인, 고친 방법과 고친 뒤 결과

발표와 주행 영상(10점): 5주차에 발표합니다. 주행 영상은 미션이 있는 판을 처음부터 끝까지 녹화한 것 1편이며, 시드를 표시합니다.

3.6 재채점

- 시뮬레이터나 브릿지 문제로 판이 깨지면 같은 시드로 다시 돌립니다.
- 여러분 노드가 죽거나 멈춰서 생긴 결과는 그대로 채점합니다.
- 패키지 이름이나 의존성 누락처럼 로직과 무관한 실행 오류는 한 번에 한해 고칠 기회를 줍니다. 이때 주행 점수에서 10점을 깎습니다.

4 제출과 규칙

4.1 제출물

다음 세 가지를 제출합니다.

1. **`team_XX.zip`** : `ros2_ws/src/team_XX` 폴더 통째로. 압축을 풀면 `team_XX/package.xml` 이 바로 나와야 합니다.
   - `ros2 launch team_XX drive.launch.py` 한 줄로 실행되어야 합니다.
   - Python 패키지가 더 필요하면 `team_XX/requirements.txt` 에 버전과 함께 적습니다(예: `ultralytics==8.3.0`). 시스템 Python 3.10 기준입니다.
   - 모델 가중치는 패키지 안에 넣습니다(전체 500 MB 이하). `setup.py` 의 `data_files` 에 넣고 `get_package_share_directory("team_XX")` 로 경로를 찾으세요.
   - README에 의존성과 실행 방법을 적습니다.
2. **보고서 PDF**
3. **주행 영상** (또는 영상 링크)

> **실행 중에는 아무것도 내려받으면 안 됩니다**
> 교수자는 `requirements.txt` 설치까지만 네트워크를 씁니다. 가중치가 없을 때 자동으로 받는 코드(예: `YOLO("yolov8s.pt")` 처럼 파일 경로 대신 이름만 주는 것)는 채점에서 실패할 수 있습니다.

4.2 제출 전 확인

새 터미널에서 아래 순서대로 했을 때 문제없이 돌아야 합니다.

- ☐ 깨끗한 워크스페이스에 `team_XX.zip` 을 풉니다.
- ☐ `colcon build` 를 실행합니다(`--symlink-install` 없이).
- ☐ 가상환경에 `pip install -r requirements.txt` 를 합니다.
- ☐ `./scripts/grade.sh --launch "<실행 명령>" <시드>` 로 한 판을 끝까지 돌립니다.

4.3 규칙

구분	내용
써도 되는 것	오픈소스 코드, 사전학습 모델(COCO 등), 시뮬레이터 영상으로 직접 만든 데이터로 학습·파인튜닝, 생성형 AI 도구. 보고서에 **출처와 사용 범위**를 밝혀야 합니다
안 되는 것	다른 팀과 코드 공유 · 시뮬레이터 파일을 고치는 것
공유해도 되는 것	시드와 결과. 채점 시드는 비공개입니다

부록 파일 위치

저장소 폴더(`avsim-assignment/`) 기준 상대 경로입니다. `~` 는 홈 폴더입니다.

무엇	경로
팀 패키지	`ros2_ws/src/team_XX/`
주행 코드 (채울 곳)	`ros2_ws/src/team_XX/team_XX/driver.py`
제출할 실행 파일	`ros2_ws/src/team_XX/launch/drive.launch.py`
시뮬레이터 받기 · 실행	`scripts/get-sim.sh` · `scripts/run-sim.sh`
채점 스크립트	`scripts/grade.sh`
결과 요약	`scripts/scenario_average.py`
카메라 영상 저장	`scripts/record_frames.py`
플레이어 로그 (시드 확인)	`~/.config/unity3d/DefaultCompany/UnitySim/Player.log`
결과 파일 (`--result` 를 안 줬을 때)	`~/.config/unity3d/DefaultCompany/UnitySim/scenario_result.json`
`grade.sh` 결과	`/tmp/avsim-grade-<날짜-시각>/`
요약 · 자세한 안내서	`README.md` · `docs/assignment/student-guide.md`
과제 버전	`VERSION`